

Min number of days to make m bouquets

Companies: Adobe, Navi

Problem Statement

given → a vector of n flowers, which represents the flowers of garden, its value represent days in which the (flower_i) will bloom

We need to make m bouquets and to make one bouquet we need k adjacent flowers from garden

Example Walkthrough

Let bloom array be: `[7, 7, 7, 7, 12, 7, 7]`, $m=2, k=3$

`[7, 7, 7, 7]`

↑
first bouquet
after 7 days

`[12, 7, 7]`

↑
second bouquet
after 12 days

if day is enough to make m bouquets
then day+1, day+2, are also enough

so Binary search

Binary Search Bounds & Logic

$h = \text{*max_element}$ → let all grow

$l = 1$ → atleast ek to ugne do

Simulation:

let `day = 8` then we make a bloom array

```
[1, 1, 1, 1, 0, 1, 1]
```

↑ not yet grown

and traverse it to make bouquets

- so `[1, 1, 1]` → made bouquet++ ; count = 0
- we increase count only if `arr[i] == 1`
- if `arr[i] == 0` then `count = 0`
- if `arr[i] == 1` then `count++` , and if `count == k` (bouquet++)

C++ Implementation

```
class Solution {
public:
    int n;
    bool check(int day, vector<int>& bloom, int m, int k) {

        vector<int> status(n, 0);
        for(int i = 0; i < n; i++) {
            if(day >= bloom[i]) {
                status[i] = 1;
            }
        }

        int bouquet = 0;
        int currcount = 0;

        for(int i = 0; i < n; i++) {
            if(status[i] == 1) {
                currcount++;
            } else {
                currcount = 0;
            }

            if(currcount == k) {
                bouquet++;
                currcount = 0;
            }
        }
    }
};
```

```

        if(bouquet >= m) return true;
    }
    return false;
}

int minDays(vector<int>& bloomDay, int m, int k) {
    n = bloomDay.size();

    int l = *min_element(bloomDay.begin(), bloomDay.end());
    int h = *max_element(bloomDay.begin(), bloomDay.end());

    int ans = -1;
    while(l <= h) {
        int mid = l + (h - l) / 2;
        if(check(mid, bloomDay, m, k)) {
            ans = mid;
            h = mid - 1;
        } else {
            l = mid + 1;
        }
    }

    return ans;
}
};

```